

Day 02

Date: 2026-09-17

Topic: Python for AI Engineering

What I Learned

- Learned how variables are used to store and manage data in Python.
- Learned the differences between common Python data types such as strings, integers, floats, and booleans.
- Learned how to use lists, dictionaries, and sets to store and organize data.
- Learned how if, elif, and else are used to make decisions in a program.
- Learned how for and while loops are used to repeat operations.
- Learned how functions help organize reusable pieces of code.
- Learned the basic concept of classes and objects in Python.
- Learned how Python modules can be used to separate code into different files.
- Learned how virtual environments isolate project dependencies.
- Learned how pip is used to install Python packages.
- Learned the purpose of requirements.txt for managing project dependencies.
- Learned how exception handling with try and except prevents unexpected input from crashing a program.
- Learned how JSON is used to store and exchange structured data.
- Learned how environment variables can be used to store configuration separately from source code.
- Learned why .env files should not be committed to GitHub when they contain sensitive configuration or credentials.

Attention- (this content copied from chatgpt, according to the simple project that I did)

What I Implemented

- Created the Week-01/Day-02 project structure in the training repository.
- Created and used a Python virtual environment for the project.
- Created a requirements.txt file to record the project's Python dependencies.
- Created a .gitignore file to exclude .venv, Python cache files, and .env.
- Created .env and .env.example files for environment-based configuration.
- Implemented configuration loading using python-dotenv.
- Built a Python program that accepts user information such as name, age, and role.
- Used dictionaries to represent individual users.
- Used lists to store multiple users.
- Used sets to identify unique user roles.
- Used conditions to categorize users into age groups.
- Used loops to process multiple users.
- Used functions to organize input validation and processing logic.
- Used a class to organize user-processing functionality.
- Saved processed user information into a JSON file.
- Added exception handling to manage invalid user input.
- Tested the program using both valid and invalid inputs.

What Worked

- The virtual environment was created and activated successfully.
- The required Python package was installed successfully.
- The application successfully loaded configuration values from the .env file.
- The program successfully accepted user input from the terminal.
- Multiple users could be entered and stored in a list.
- User information was successfully represented using dictionaries.

- The program successfully classified users according to their age.
- Unique user roles were successfully identified using a set.
- Processed data was successfully saved to data.json.
- Invalid numeric input was handled without crashing the application.
- The project files were organized into separate modules instead of keeping all the code in one file.
- The project was successfully committed to Git.

```

PS E:\VisionRoot-AI engineer Internship\GitHub\AI-Engineer-Training\AI-Engineer-Training\Week-01\Day-02> python -m venv .venv
PS E:\VisionRoot-AI engineer Internship\GitHub\AI-Engineer-Training\AI-Engineer-Training\Week-01\Day-02> .\.venv\Scripts\Activate.ps1
(.venv) PS E:\VisionRoot-AI engineer Internship\GitHub\AI-Engineer-Training\AI-Engineer-Training\Week-01\Day-02> python -m pip install python-dotenv
Collecting python-dotenv
  Downloading python_dotenv-1.2.3-py3-none-any.whl.metadata (29 kB)
  Downloading python_dotenv-1.2.3-py3-none-any.whl (22 kB)
Installing collected packages: python-dotenv
Successfully installed python-dotenv-1.2.3

[notice] A new release of pip is available: 26.1.2 -> 26.2.1

(.venv) PS E:\VisionRoot-AI engineer Internship\GitHub\AI-Engineer-Training\AI-Engineer-Training\Week-01\Day-02> python -m pip install python-dotenv
[notice] A new release of pip is available: 26.1.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS E:\VisionRoot-AI engineer Internship\GitHub\AI-Engineer-Training\AI-Engineer-Training\Week-01\Day-02> python -m pip freeze
python-dotenv==1.2.3
(.venv) PS E:\VisionRoot-AI engineer Internship\GitHub\AI-Engineer-Training\AI-Engineer-Training\Week-01\Day-02> python -m pip freeze > requirements.txt

```

What Failed

- Initially, converting user input directly to an integer caused a ValueError when non-numeric input was entered.
- The program initially did not handle invalid input properly and could terminate with an error.
- I initially had difficulty understanding why environment variables are stored as strings and need to be converted when numerical values are required.

How I Solved It

- Investigated the error message and identified ValueError as the result of trying to convert invalid text into an integer.
- Used try and except ValueError to handle invalid numerical input safely.
- Added conditional checks to prevent logically invalid values such as negative ages.

- Used `os.getenv()` to retrieve environment variables and converted numerical configuration values using `int()`.
- Used `.env.example` as a safe template while keeping the actual `.env` file excluded through `.gitignore`.
- Separated configuration and user-processing logic into different Python modules and imported the required values and functions where needed.
- Tested the application repeatedly with both valid and invalid inputs to verify that the error-handling logic worked correctly.
- Used terminal output and error messages to investigate problems rather than ignoring the errors.

```
(.venv) PS E:\VisionRoot-AI engineer Internship\GitHub\AI-Engineer-Training\AI-Engineer-Training\Week-01\Day-02> python main.py
Enter Your Name: fdhfhf
fdhfhf
Enter Your Age: eh
Traceback (most recent call last):
  File "E:\VisionRoot-AI engineer Internship\GitHub\AI-Engineer-Training\AI-Engineer-Training\Week-01\Day-02\main.py", line 4, in <module>
    age = int(input("Enter Your Age: "))
ValueError: invalid literal for int() with base 10: 'eh'
(.venv) PS E:\VisionRoot-AI engineer Internship\GitHub\AI-Engineer-Training\AI-Engineer-Training\Week-01\Day-02> python main.py
Enter Your Name: dfsg
dfsg
Enter Your Age: 34
34
(.venv) PS E:\VisionRoot-AI engineer Internship\GitHub\AI-Engineer-Training\AI-Engineer-Training\Week-01\Day-02> python main.py
```

What I Would Improve

- Add automated unit tests for the input validation and user-processing functions.
- Improve the error messages to make them more informative for users.
- Add Python type hints to make the code easier to understand and maintain.
- Add logging instead of relying only on `print()` statements.
- Improve the project structure as the application becomes larger.
- Add more robust validation for names, ages, and roles.
- Improve the JSON data storage process for larger amounts of data.
- Add a proper command-line interface in a future version.
- Connect the application to a database instead of using only JSON for persistent storage.

- Add documentation explaining how another developer can install and run the project.